Robotics Lab

KUKA iiwa Trajectory Planning and Inverse Dynamics

Federico Trenti

Academic year 2024/2025

Project Repository

GitHub repository

Contents

1 **4**

1.1 Substitution of the current trapezoidal velocity profile with a cubic polinomial linear trajectory 4

2 **7**

2.1 Creation of circular trajectories for the robot 7

3 **10**

3.1 Test of the trajectories 10

4 **16**

4.1 Develop an inverse dynamics operational space controller . . 16

4.2 Plot of the torques in the operational space for the given trajectories 19

Chapter 1

1.1 Substitution of the current trapezoidal velocity profile with a cubic polinomial linear trajectory

First of all we downloaded the "ros2_kdl_package" package from the Github repository.

```
$ git clone https://github.com/RoboticsLab2024/
   ros2_kdl_package.git
```

After that we modified appropriately the KDLPlanner class, inside the files "kdl_planner.h" and "kdl_planner.cpp", in order to define a new `KDLPlanner::trapezoidal_vel` function that takes the current time t and the acceleration time t_c as double arguments and returns three double variables s, $\dot{s}$ and $\ddot{s}$ that represent the curvilinear abscissa of our trajectory. Considering that a trapezoidal velocity profile for a curvilinear abscissa $s \in [0, 1]$ is defined as:

$$s(t) = \begin{cases} \frac{1}{2} s_c t^2 & \text{se } 0 \leq t \leq t_c, \\ s_c t_c (t - t_c/2) & \text{se } t_c < t \leq t_f - t_c, \\ 1 - \frac{1}{2} s_c (t_f - t)^2 & \text{se } t_f - t_c < t \leq t_f. \end{cases}$$

```
//Definition of the prototype in kdl_planner.h
void trapezoidal_vel(double t, double tc, double& s,
   double& s_dot, double& s_ddot);
```

```
//Implementation in kdl_planner.cpp
void KDLPlanner::trapezoidal_vel(double time, double
   accDuration, double& s, double& s_dot, double&
   s_ddot){
```

```
  /* trapezoidal velocity profile with accDuration
     acceleration time period and trajDuration_ total
     duration.
     time = current time
     trajDuration_  = final time
     accDuration   = acceleration time
     trajInit_ = trajectory initial point
     trajEnd_  = trajectory final point */

  double sc_ddot=-1.0/(std::pow(accDuration,2)-
     trajDuration_*accDuration);
  if(time <= accDuration)
  {
    s = 0.5*sc_ddot*std::pow(time,2);
    s_dot = sc_ddot*time;
    s_ddot = sc_ddot;
  }
  else if(time <= trajDuration_-accDuration)
  {
    s = sc_ddot*accDuration*(time-(accDuration/2));
    s_dot = sc_ddot*accDuration;
    s_ddot = 0.0;
  }
  else
  {
    s = 1 - 0.5*sc_ddot*std::pow(trajDuration_-time,2);
    s_dot = sc_ddot*(trajDuration_-time);
    s_ddot = -sc_ddot;
  }
}
```

From there, we had to create another function named `KDLPlanner::cubic_polynomial` that creates the cubic polynomial curvilinear abscissa for our trajectory. In this case, we considered the formula:

$$s(t) = a_3 t^3 + a_2 t^2 + a_1 t + a_0$$

So we added in both previous files the prototype and its implementation.

```
//Definition of the prototype in kdl_planner.h
void cubic_polynomial(double t, double& s, double&
   s_dot, double& s_ddot);
```

```
//Implementation in kdl_planner.cpp
void KDLPlanner::cubic_polynomial(double t, double& s,
   double& s_dot, double& s_ddot){
```

```
3   double a0, a1, a2, a3, s0, sDot0, sDotf, sf; //
       Coefficienti del polinomio cubico
4   //condizioni iniziali
5   s0 = 0;
6   sDot0 = 0;
7   sDotf = 0;
8   sf = 1;
9   // Calcola i coefficienti del polinomio cubico
10  a0 = s0;
11  a1 = sDot0;
12  a2 = (3 * (sf - s0) / (trajDuration_ * trajDuration_)
       ) - ((2 * sDot0 + sDotf) / trajDuration_);
13  a3 = (-2 * (sf - s0) / (trajDuration_ * trajDuration_
        * trajDuration_)) + ((sDot0 + sDotf) / (
       trajDuration_ * trajDuration_));
14
15  // Calcola s(t), s'(t) e s''(t) usando i coefficienti
        a0, a1, a2, a3
16  s = a3 * t * t * t + a2 * t * t + a1 * t + a0;
17  s_dot = 3 * a3 * t * t + 2 * a2 * t + a1;
18  s_ddot = 6 * a3 * t + 2 * a2;
19 }
```

The function takes as argument a double t representing time and returns three double s, $\dot{s}$ and $\ddot{s}$ that represent the curvilinear abscissa of our trajectory.

Chapter 2

2.1 Creation of circular trajectories for the robot

We began by creating a new constructor `KDLPlanner::KDLPlanner` that takes as arguments the time duration, the starting point and the radius of our trajectory and store them in the corresponding class variables.
As before, we updated "kdl_planner.h" and "kdl_planner.cpp".

```
//Definition in kdl_planner.h
KDLPlanner(double _trajDuration, Eigen::Vector3d
   _trajInit, double _trajRadius);
```

```
//Implementation in kdl_planner.cpp
KDLPlanner::KDLPlanner(double _trajDuration, Eigen::
   Vector3d _trajInit, double _trajRadius)
{
    trajDuration_ = _trajDuration;
    trajInit_ = _trajInit;
    trajRadius_ = _trajRadius;
    trajEnd_ = Eigen::Vector3d::Zero();
}
```

In order to obtain the desidered trajectories, we implemented the positional path directly in the function `KDLPlanner::compute_trajectory` and in `KDLPlanner::compute_trajectory_circ`.

```
//Computing trajectories for rectilinear path and
   trapezoidal
trajectory_point KDLPlanner::compute_trajectory(double
   time, double accDuration)
{
  double s,s_dot,s_ddot;
  this->trapezoidal_vel(time, accDuration, s, s_dot,
     s_ddot);
```

```
  trajectory_point traj;

  traj.pos = trajInit_ + s*(trajEnd_-trajInit_);
  traj.vel = s_dot*(trajEnd_-trajInit_);
  traj.acc = s_ddot*(trajEnd_-trajInit_);
  return traj;

}

//Computing trajectories for rectilinear path and
   cubic_polynomial
trajectory_point KDLPlanner::compute_trajectory(double
   time)
{
  double s,s_dot,s_ddot;
  this->cubic_polynomial(time, s, s_dot, s_ddot);
  trajectory_point traj;

  traj.pos = trajInit_ + s*(trajEnd_-trajInit_);
  traj.vel = s_dot*(trajEnd_-trajInit_);
  traj.acc = s_ddot*(trajEnd_-trajInit_);
  return traj;

}
```

```
//Computing trajectories for circular path and
   trapezoidal
trajectory_point KDLPlanner::compute_trajectory_circ(
   double time, double accDuration)
{
  double s,s_dot,s_ddot;
  this->trapezoidal_vel(time, accDuration, s, s_dot,
     s_ddot);
  trajectory_point traj;

  traj.pos.x() = trajInit_.x();
  traj.pos.y() = trajInit_.y() - trajRadius_*cos(2*M_PI
     *s);
  traj.pos.z() = trajInit_.z() - trajRadius_*sin(2*M_PI
     *s);
  traj.vel.x() = 0;  //s_dot*(trajEnd_-trajInit_);
  traj.vel.y() = trajRadius_*sin(2*M_PI*s)*2*M_PI*s_dot
     ;
  traj.vel.z() = -trajRadius_*cos(2*M_PI*s)*2*M_PI*
     s_dot;
  traj.acc.x() = 0;
```

```
15    traj.acc.y() = -trajRadius_*2*M_PI*(-2*M_PI*sin(2*
          M_PI*s))*pow(s_dot,2)+cos(2*M_PI*s)*s_ddot;
16    traj.acc.z() = -trajRadius_*2*M_PI*(-trajRadius_*sin
          (2*M_PI*s)*pow(s_dot,2)+cos(2*M_PI*s)*s_ddot);
17
18    //traj.acc = s_ddot*(trajEnd_-trajInit_);
19    return traj;
20
21  }
22
23
24  //Computing trajectories for circular path and
        trapezoidal
25  trajectory_point KDLPlanner::compute_trajectory_circ(
        double time)
26  {
27    double s,s_dot,s_ddot;
28    this->cubic_polynomial(time, s, s_dot, s_ddot);
29    trajectory_point traj;
30
31    traj.pos.x() = trajInit_.x();
32    traj.pos.y() = trajInit_.y() - trajRadius_*cos(2*M_PI
          *s);
33    traj.pos.z() = trajInit_.z() - trajRadius_*sin(2*M_PI
          *s);
34    traj.vel.x() = 0;  //s_dot*(trajEnd_-trajInit_);
35    traj.vel.y() = trajRadius_*sin(2*M_PI*s)*2*M_PI*s_dot
          ;
36    traj.vel.z() = -trajRadius_*cos(2*M_PI*s)*2*M_PI*
          s_dot;
37    traj.acc.x() = 0;
38    traj.acc.y() = -trajRadius_*2*M_PI*(-2*M_PI*sin(2*
          M_PI*s))*pow(s_dot,2)+cos(2*M_PI*s)*s_ddot;
39    traj.acc.z() = -trajRadius_*2*M_PI*(-trajRadius_*sin
          (2*M_PI*s)*pow(s_dot,2)+cos(2*M_PI*s)*s_ddot);
40
41    //traj.acc = s_ddot*(trajEnd_-trajInit_);
42    return traj;
43
44  }
```

Chapter 3

3.1 Test of the trajectories

Considering the trajectories obtained in the previous chapter, we tested them with the joint space inverse dynamic controller. Before plotting, the torques sent to the manipulator, we tuned appropriately the gains K_p and K_d to reach a smooth behavior.
The function that return the torques by taking as inputs the desire joints position,velocity and acceleration:

```
1     Eigen::VectorXd KDLController::idCntr(KDL::JntArray
          &_qd,
2                                           KDL::JntArray &
                                               _dqd,
3                                           KDL::JntArray &
                                               _ddqd,
4                                           double _Kp,
                                               double _Kd)
5 {
6     // read current state
7     Eigen::VectorXd q = robot_->getJntValues();
8     Eigen::VectorXd dq = robot_->getJntVelocities();
9
10    // calculate errors
11    Eigen::VectorXd e = _qd.data - q;
12    Eigen::VectorXd de = _dqd.data - dq;
13
14    Eigen::VectorXd ddqd = _ddqd.data;
15    return robot_->getJsim() * (ddqd + _Kd*de + _Kp*e)
16            + robot_->getCoriolis(); //+ robot_->
                getGravity() /*friction compensation?*/;
                   // gravity has been set to 0 in the .
                world
17 }
```

As follow, the node's implementation is:

```
    //  JOINT SPACE INVERSE DYNAMICS CONTROL

                    // Compute differential IK

                    joint_velocities_old_.data =
                       joint_velocities_.data;

                    Vector6d cartvel; cartvel << p.vel
                       + 5*error, o_error;
                    joint_velocities_.data =
                       pseudoinverse(robot_->
                       getEEJacobian().data)*cartvel;
                    joint_positions_.data =
                       joint_positions_.data +
                       joint_velocities_.data*dt;
                    joint_accelerations_d_.data = (
                       joint_velocities_.data -
                       joint_velocities_old_.data)/dt;


                    KDLController controller_(*robot_);
                    joint_efforts_.data = controller_.
                       idCntr(joint_positions_,
                       joint_velocities_,
                       joint_accelerations_d_, 50.0,
                       5.0);

                    KDL::Frame frame_final = robot_->
                       getEEFrame();
                    KDL::Twist velocities_final;
                       velocities_final.vel=KDL::Vector
                       ::Zero(); velocities_final.rot=
                       KDL::Vector::Zero();
                    KDL::Twist acceleration_final;
                       acceleration_final.vel=KDL::
                       Vector::Zero();
                       acceleration_final.rot=KDL::
                       Vector::Zero();
```

Moreover, to ensure Gazebo starts at launch the default value of "use_sim" has been set to true in both the "iiwa.launch.py" file and the "iiwa.config.xacro" file.

```
<xacro:arg name="use_sim" default="true" />
```

```
declared_arguments.append(
        DeclareLaunchArgument(
            'use_sim',
            default_value='true',
            description='Start robot in Gazebo
               simulation.',
        )
    )
```

To run our world, it has been added in "iiwa.launch.py" the following code:

```
declared_arguments.append(
        DeclareLaunchArgument(
            'gz_args',
            default_value=[iiwa_simulation_world, ' -r
               -v 1'],
            description='Arguments for gz_sim'
        )
    )
```

The effort_controller has added inside the file "iiwa_controllers.yaml":

```
    effort_controller:
      type: effort_controllers/
         JointGroupEffortController
```

```
effort_controller:
  ros__parameters:
    command_interfaces:
      - effort
    state_interfaces:
      - position
      - velocity
      - effort
    joints:
      - joint_a1
      - joint_a2
      - joint_a3
      - joint_a4
      - joint_a5
      - joint_a6
      - joint_a7

    state_publish_rate: 200.0
    action_monitor_rate: 20.0
```

In the file "empty.world" has been added two links from internet:

```
    <include>
      <uri>https://fuel.gazebosim.org/1.0/OpenRobotics/
         models/Ground Plane</uri>
    </include>
    <include>
      <uri>https://fuel.gazebosim.org/1.0/OpenRobotics/
         models/Sun</uri>
    </include>
```

To visualize them it's possible to use the command:

```
$ ros2 run rqt_plot rqt_plot
```

Then, we selected `/effort_controller/commands/data[i]` for all the seven joints.

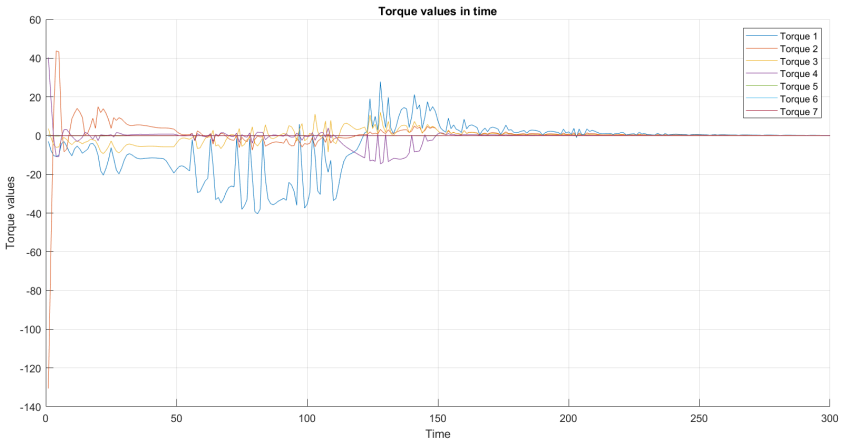


Figure 3.1: Torques values in joints space for a linear trapezoidal trajectory

After that, we created a script in MATLAB which receive a bag file that contains the informations about the torques of each joint. In order to create this bag file we used the command:

```
$ ros2 bag record effort_controller/commands/data -
   o my_bag
```

This bag file was sent to a MATLAB script to retrieve the desidered plot of the torques. This file is available in the repository.

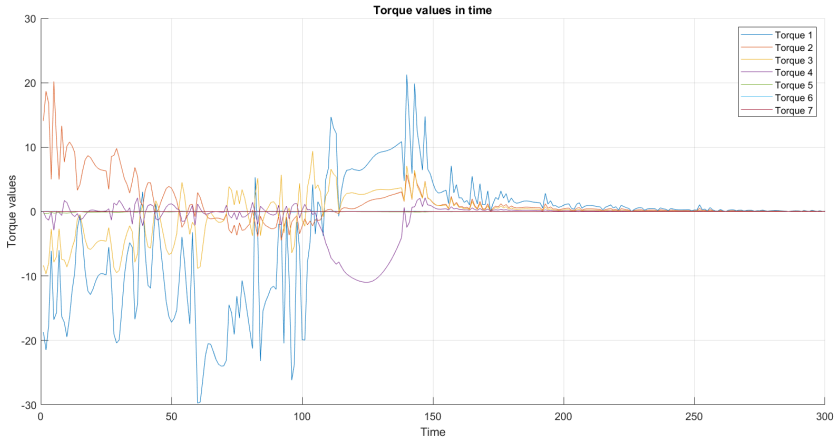


Figure 3.2: Torques values in joints space for a linear cubic_polynomial trajectory

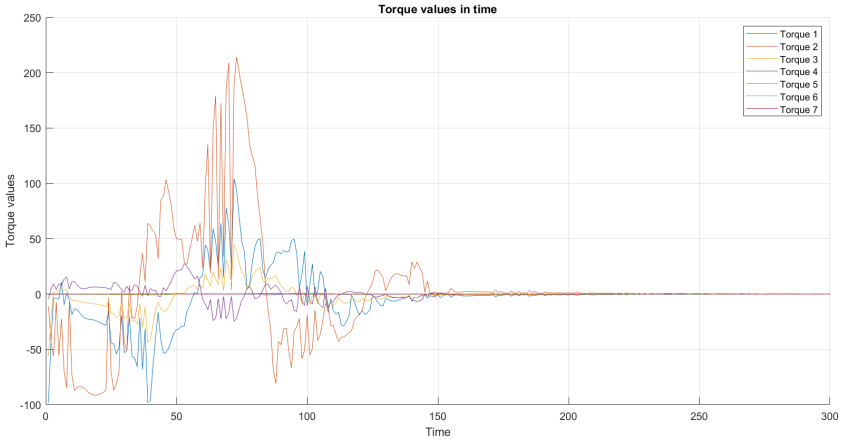


Figure 3.3: Torques values in joints space for a circular trapezoidal trajectory

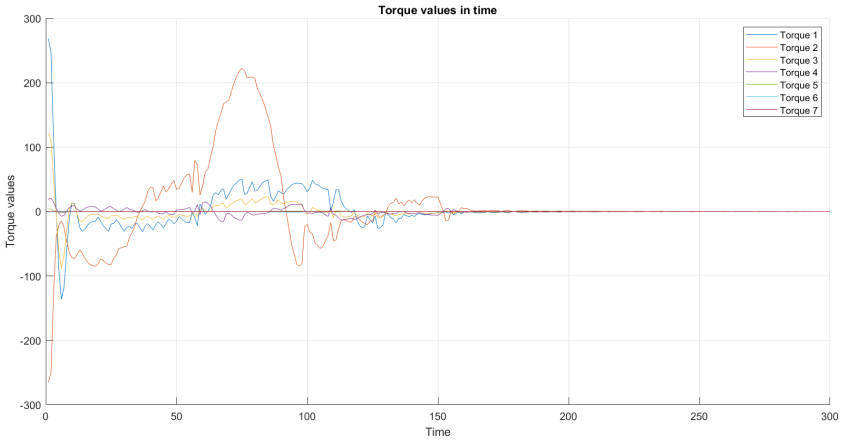


Figure 3.4: Torques values in joints space for a circular cubic_polynomial trajectory

Chapter 4

4.1 Develop an inverse dynamics operational space controller

Then the four trajectories have been implemented with the operational space inverse dynamic controller tuning appropriately the gains Kpp, Kpo, Kdp, Kdo to reach a smooth behaviour and making sure the manipulator stays at rest after completing the trajectory. The operational space controller function takes as inputs the desired frame, velocity and acceleration and the four gains and returns the torque. The operational space controller has been tested on the four trajectories and the joint torque commands has been plot. The code is :

```
1     Eigen::VectorXd KDLController::idCntr(KDL::Frame &
          _desPos,
2                                           KDL::Twist &
                                                _desVel,
3                                           KDL::Twist &
                                                _desAcc,
4                                           double _Kpp,
                                                double _Kpo,
5                                           double _Kdp,
                                                double _Kdo)
6 {
7     // read current state
8     KDL::Frame x = robot_->getEEFrame();
9     KDL::Twist dx = robot_->getEEVelocity();
10
11    Vector6d x_tilde, dot_x_tilde;
12
13    Eigen::Matrix<double,6,6> Kp;
14    Eigen::Matrix<double,6,6> Kd;
15    Kp.setZero();
16    Kd.setZero();
17
```

```
    Kp.block(0,0,3,3) = _Kpp*Eigen::Matrix3d::Identity
       ();
    Kp.block(3,3,3,3) = _Kpo*Eigen::Matrix3d::Identity
       ();
    Kd.block(0,0,3,3) = _Kdp*Eigen::Matrix3d::Identity
       ();
    Kd.block(3,3,3,3) = _Kdo*Eigen::Matrix3d::Identity
       ();

    computeErrors(_desPos,x,_desVel,dx,x_tilde,
       dot_x_tilde);

    for (unsigned int i=0;i<3;i++){x_tilde(i)=_Kpp*
       x_tilde(i);dot_x_tilde(i)=_Kdp*dot_x_tilde(i);}
    for (unsigned int i=3;i<6;i++){x_tilde(i)=_Kpo*
       x_tilde(i);dot_x_tilde(i)=_Kdo*dot_x_tilde(i);}

    Eigen::Matrix<double,7,1> y;
    y = pseudoinverse(robot_->getEEJacobian().data)*(
       toEigen(_desAcc)+x_tilde+dot_x_tilde-robot_->
       getEEJacDotqDot());

    return  robot_->getJsim()*y + robot_->getCoriolis()
       ; //+ robot_->getGravity();      // gravity has
       been set to 0 in the .world
}
```

```
// OPERATIONAL SPACE INVERSE DYNAMICS CONTROL

                    Kpp = 250.0;
                    Kpo = 250.0;
                    Kdp = 50.0;
                    Kdo = 50.0;

                    KDL::Frame desired_frame;
                    desired_frame.p = toKDL(p.pos);
                    desired_frame.M = cartpos.M;

                    KDL::Twist desired_vel;
                    desired_vel.vel = toKDL(p.vel);

                    KDL::Twist desired_accel;
                    desired_accel.vel = toKDL(p.acc);

                    robot_->getInverseKinematics(
                       desired_frame,
```

```
                    des_joint_positions_);
19                  robot_->getInverseKinematicsVel(
                    desired_vel,
                    des_joint_velocities_);
20                  robot_->getInverseKinematicsAcc(
                    desired_accel,
                    joint_accelerations_d_);  //
                    defined in utils.h
21
22                  joint_efforts_.data=controller_.
                    idCntr(desired_frame,desired_vel
                    ,desired_accel,Kpp, Kpo, Kdp,
                    Kdo);
23              }
```

To stop the manipulator, the code is:

```
1   if(cmd_interface_ =="effort")
2                   {
3                       KDLController controller_(*robot_);
4                       des_joint_velocities_.data=Eigen::
                        VectorXd::Zero(7,1);
5                       des_joint_accelerations_.data=Eigen
                        ::VectorXd::Zero(7,1);
6
7                       joint_efforts_.data=controller_.
                        KDLController::idCntr(
                        joint_positions_,
                        des_joint_velocities_,
                        des_joint_accelerations_, 50.0,
                        5.0);
8
9                       robot_->update(toStdVector(
                        joint_positions_.data),
                        toStdVector(joint_velocities_.
                        data));
10
11                      for (long int i = 0; i <
                        joint_velocities_.data.size();
                        ++i) {
12                       desired_commands_[i] =
                            joint_efforts_.data[i];
13                      }
14                  }
```

4.2 Plot of the torques in the operational space for the given trajectories

As follow, the plot of the torques in the operational space:

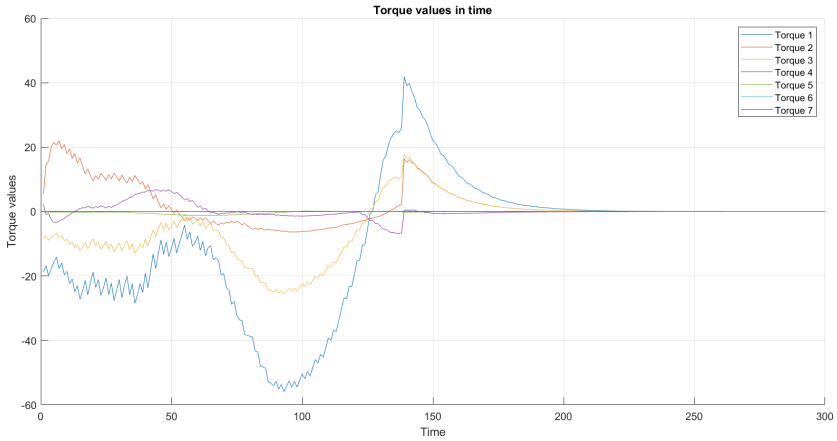


Figure 4.1: Torques values in operational space for a linear trapezoidal trajectory

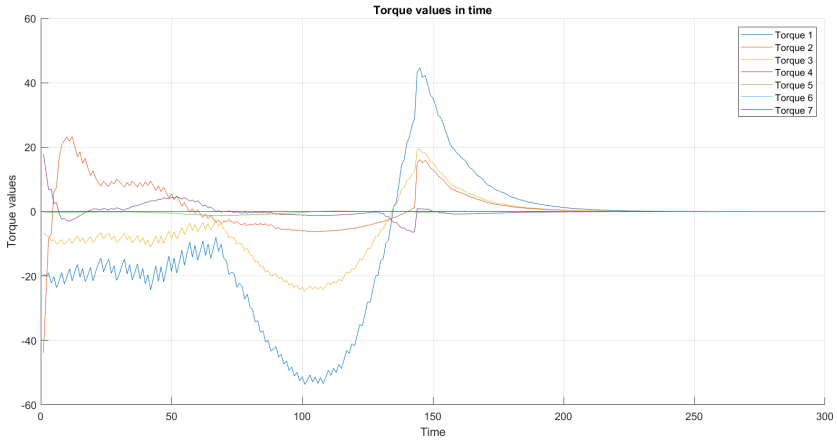


Figure 4.2: Torques values in operational space for a linear cubic_polynomial trajectory



Figure 4.3: Torques values in operational space for a circular trapezoidal trajectory

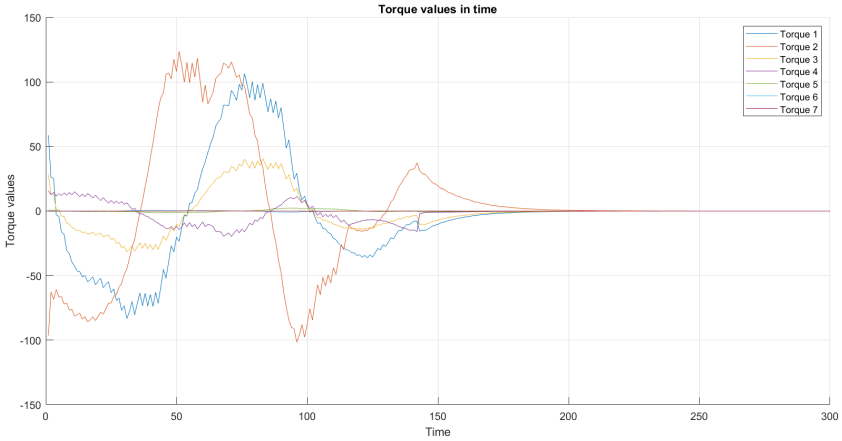


Figure 4.4: Torques values in operational space for a circular cubic_polynomial trajectory